

JEREMI AXCEL CORTEZ HUIJAR [22200013]

- I.1. El repositorio se considera la única fuente de verdad porque centraliza toda la lógica de acceso a datos, ocultando completamente a la UI si los datos provienen de Room o de Retrofit. Actúa como intermediario que coordina la sincronización entre ambos permitiendo que la UI nunca conozca de donde exactamente vienen los datos ni como se obtienen.
2. El viewModelScope es un scope de corrutinas que está vinculado directamente al ciclo de vida del ViewModel, siendo importante en la gestión de memoria porque cancela automáticamente todas las corrutinas cuando el ViewModel se destruye, previniendo memory leaks. Cuando el usuario decide cerrar la pantalla antes de recibir la respuesta de una petición de red iniciada en este scope, la corrutina se cancela automáticamente, evitando que siga consumiendo recursos.
3. Un Flow solo emite valores cuando hay colectores suscritos, sin mantener estado previo y empezando la emisión desde cero, mientras que un stateFlow siempre está emitiendo, mantenga o no colectores activos, y almacena el último valor emitido para entregarlo inmediatamente a los nuevos suscriptores. Para la UI se prefiere stateFlow porque se prefiere que la interfaz conozca el estado actual.

- II. 5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama `callbackFlow`.
6. El archivo de metadatos que describe la arquitectura de la App y el sistema operativo se llama exactamente `AndroidManifest.xml`.
7. En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL.
8. El operador de Flow que combina múltiples fuentes como (GPS y sensores) para emitir un estado unificado, es `combine`.

```
III. 1. #class UsuarioRepository (private val usuarioDAO : UsuarioDAO) {
    fun obtenerUsuarios () = usuarioDAO.getAllUsuarios ()
}
```

```
# class MyApplication : Application () {
    val usuarioRepository : UsuarioRepository by lazy {
        UsuarioRepository (AppDatabase.getInstance (applicationContext).
            usuarioDAO ())
    }
}
```


I. 4. La DI se implementa creando las instancias singleton de componentes como la base de datos y el repositorio dentro de la clase Application, para que toda la app comparta los mismos objetos.

La palabra by lazy sirve para que estos objetos se inicialicen una sola vez, cuando se necesiten, mejorando el rendimiento.

II. 1. Flow

2.

3.

4.

5.

6. AndroidManifest.xml

7.

8.

II. 1. El flujo de datos que emite automáticamente una tabla de Room cambia se implementa usando el tipo Flow

2. El operador dispatchers permite ejecutar tareas pesadas fuera del main thread, específicamente optimizado para tareas de CPU como el parseo masivo de datos

3. Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo Foreground-service

4. La función getInstance() de la clase Application es la que garantiza que los componentes comparten la misma instancia del repositorio mediante un "cast."

III. 2. class Usuario {

```
val nombre : String?  
val apellidos : String?  
val edad : Int?
```

}

```
val yo = Usuario("Jeremi", "Cortez", 25)
```

```
fun Puntaje (usuario : Usuario) : Int {
```

```
    return (usuario.nombre?.length ?: 0) + (usuario.apellidos?.length ?: 0)
```

```
}
```

```
// Val puntos = Puntaje (yo)
```

```
println ("Usuario: ${yo.nombre} ${yo.apellidos}, tu puntaje de  
identidad es $puntos")
```

IV. 1.

Aplicaciones
FRAMEWORK
Bibliotecas y ART
HAL
Kernel de Linux

2. • Aplicaciones. Contiene las apps del usuario y del sistema.
• Framework. conjunto de APIs y servicios que los devs usan para construir apps
• Bib y ART. Android usa bibliotecas escritas en C/C++, ART es la que ejecuta el bytecode, gestiona el recolector de basura, y usa compilación JIT/AOT para optimizar el rendimiento
• HAL. Actúa como puente entre el Hardware y Android
• Kernel. Se encarga de gestionar la comunicación entre el hardware y capas superiores.